

Text-attributed Graph Condensation via Text Selection and Attribute Matching

Haowei Han*

haowei.han@whu.edu.cn
School of Computer Science
Wuhan, China

Yuxiang Wang*

nai.yxwang@whu.edu.cn
School of Computer Science
Wuhan, China

Guojia Wan

guojiawan@whu.edu.cn
School of Computer Science
Wuhan, China

Hao Wang

wanghao.cs@whu.edu.cn
School of Computer Science
Wuhan, China

Shanshan Feng

victor_fengss@whu.edu.cn
School of Computer Science
Wuhan, China

Hao Huang

haohuang@whu.edu.cn
School of Computer Science
Wuhan, China

Jiawei Jiang[†]

jiawei.jiang@whu.edu.cn
School of Computer Science
Wuhan, China

Xiao Yan[†]

yanxiaosunny@whu.edu.cn
Institute for Math & AI
Wuhan, China

Abstract

Text-Attributed Graph (TAG) is an important type of graph structured data, where each node has a text description. TAG models usually train a Graph Neural Network (GNN) and language model jointly, which leads to high space and time consumption, especially on large datasets. To mitigate this, we propose TAGSAM, a condensation method that compresses TAGs while preserving training accuracy. TAGSAM comes with two key designs, i.e., *subgraph text selection* and *attribute similarity matching*, which compress the text description and graph topology of TAG, respectively. For the texts, subgraph text selection selects and merges representative text chunks from multiple related text descriptions by maximizing mutual information. For the graph topology, popular condensation methods based on Matching Training Trajectories (MTT) suffer from high variance, which hinders accuracy. Our attribute similarity matching mitigates this issue by aligning stable similarity matrices. We evaluate TAGSAM against six state-of-the-art baselines, where it showcases superior performance. For the same compressed size, TAGSAM improves upon the best-performing baseline by an average of 4.9% in accuracy. Furthermore, it maintains competitive training accuracy even when the TAG is condensed to just 1% size. Our code is available at <https://github.com/SundayVHan/TAGSAM>

CCS Concepts

• Information systems → Data mining.

Keywords

Text-attributed Graph, Dataset Condensation, Graph Learning

*Both authors contributed equally to this research.

[†]Corresponding authors.



This work is licensed under a Creative Commons Attribution 4.0 International License. WWW '26, Dubai, United Arab Emirates.

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2307-0/2026/04

<https://doi.org/10.1145/3774904.3792205>

ACM Reference Format:

Haowei Han, Yuxiang Wang, Guojia Wan, Hao Wang, Shanshan Feng, Hao Huang, Jiawei Jiang, and Xiao Yan. 2026. Text-attributed Graph Condensation via Text Selection and Attribute Matching. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/3774904.3792205>

1 Introduction

Text-Attributed Graph (TAG) [3, 22] is a type of multi-modal data that combine graph topology with text descriptions. Current TAG methods jointly train a Graph Neural Network (GNN) on the graph topology and a language model on the node texts. For instance, G2P2 [40] uses contrastive learning to align representations from both the graph encoder GNN and text encoder transformer [31], while GraphCLIP [48] uses a Large Language Model (LLM) to summarize a node's local neighborhood for richer context. These models are used for tasks like article classification and social network analysis [1, 12, 18, 35, 38]. By using text as an extra supervisory signal, such TAG models significantly reduce their reliance on labels compared to general graph models, demonstrating stronger zero-shot capabilities in no-label scenarios. However, this dual-modal training is memory- and time-intensive, especially on large datasets.

Dataset condensation [34, 46] is a widely used technique to reduce the training costs. In particular, a large dataset is compressed into a smaller one while maintaining competitive training accuracy. *Selective methods* [2, 7, 10, 39] curate a representative subset of the original data. For instance, Herding [39] minimizes the discrepancy in feature representations between the subset and the full dataset. *Generative methods* [34, 46, 47] learn to generate the compressed dataset by solving a bi-level optimization problem, where the inner loop trains student models on compressed dataset and the outer loop aligns student models with the teacher models, which are trained on original dataset, in a certain aspect. This principle has recently been extended to graph structured data with methods like GCond [14], which generates synthetic graphs by matching gradients between student and teacher models.

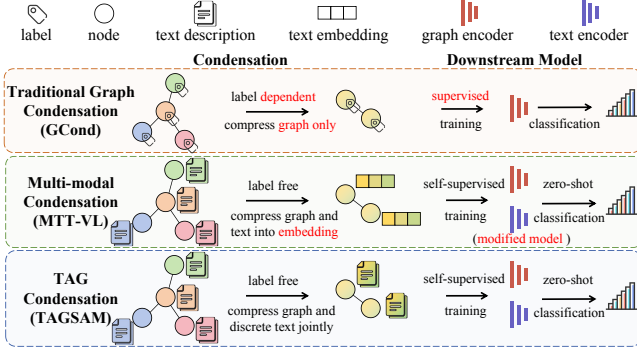


Figure 1: Comparison of different dataset condensation paradigms. Unlike existing methods, TAGSAM ensures label independence and preserves the discrete nature of text data.

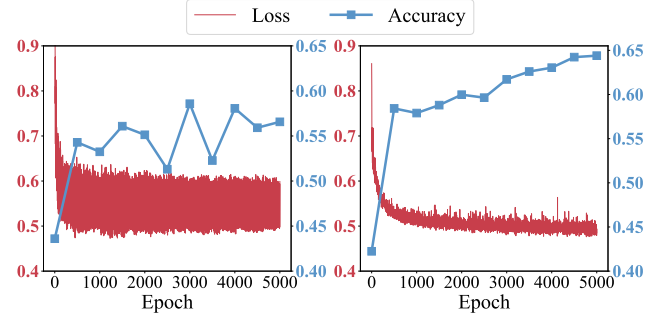
Comparison to traditional graph condensation. While effective for standard graphs, traditional graph condensation methods like GCond is ill-suited for TAGs. As Figure 1 demonstrates, such paradigm face two key limitations in this context: First, they condense only the graph modality, ignoring the rich, high-dimensional text. Second, their reliance on label supervision is incompatible with common TAG scenarios, where ground-truth labels are often scarce or entirely unavailable due to high annotation costs [40]. Consequently, these traditional methods, not being designed to preserve joint information, fail to meet the needs of TAG models for tasks like contrastive learning and zero-shot classification.

Challenges for current condensation methods. Among recent advances, the work most relevant to TAG condensation is the line of multi-modal dataset condensation methods that employ Matching Training Trajectories (MTT) to compress image-text datasets [41, 43], which involves aligning the model parameter update trajectories produced between training on original dataset and compressed one. Yet, despite their success elsewhere, we find they perform poorly on TAGs due to the following two reasons:

- **Unreadable text.** MTT employs gradient descent for condensation, which excels continuous features but struggles with discrete text, resulting in text being compressed into unreadable embedding vectors. Consequently, these embeddings are incompatible with downstream models like tokenizers. While retrieving nearest neighbor original texts in embedding space as substitutes is attempted, it incurs long runtime and lower model performance.
- **Unstable training.** As shown in Figure 2(a), MTT exhibits significantly volatile training loss and accuracy when condensing a TAG, consequently resulting in sub-optimal convergence. This instability arises because the teacher model’s contrastive loss, computed on randomly sampled mini-batches, yields high trajectory variance, and hinders the student model’s matching process.

These analysis shows that existing condensation methods cannot work well on TAGs. Thus, we ask the following research question:

Is it possible to design a TAG condensation method in no-label scenarios, which can produce discrete compressed text and achieves stable convergence?



(a) Matching Training Trajectories (b) Attribute Similarity Matching

Figure 2: Matching training trajectories vs. Attribute similarity matching. The left and right y-axes denote the model loss and accuracy, respectively.

To answer the research question, we design TAGSAM to condense TAGs, with *subgraph text selection* and *attribute similarity matching* as two key designs to tackle the two critical problems we identified for MTT. Instead of updating the text embeddings of the compressed graph with SGD, subgraph text selection directly conducts representative selection in original text space. Besides, compared to condensing TAGs via matching training trajectories, which suffers from high variance due to random batching, we propose to match attribute similarity, which is inherently more stable. This approach yields greater training stability and improves model accuracy, as shown in Figure 2(b). Furthermore, we theoretically analyze our proposed method. First, for subgraph text selection, we introduce a practical greedy method and theoretically demonstrate that it effectively approximates the optimal solution. Second, we prove that attribute similarity serves as an effective estimate for point-wise mutual information between attributes, and is thereby inherently stable, leading to more robust training.

To evaluate TAGSAM, we conduct extensive experiments and compare it with 6 state-of-the-art dataset condensation methods. The results show that TAGSAM consistently yields higher model training accuracy than all baselines. Compared with the best performing baseline, the average accuracy improvement of TAGSAM is 4.9%. Moreover, TAGSAM achieves competitive training accuracy with only a 1% condensation ratio of original TAGs. Besides, ablation studies indicate our model designs are effective. Furthermore efficiency studies suggest that TAGSAM is efficient in running time.

To summarize, we make the following contributions:

- We identify that traditional graph condensation methods are unsuitable for TAG condensation scenarios, and existing multi-modal dataset condensation methods perform poorly due to issues such as unreadable text and high variance.
- We propose subgraph text selection to select representative texts in the discrete text space, and introduce attribute similarity matching to reinforce robustness via matching stable attribute correlations instead of training trajectories.
- We conduct extensive experiments on 5 datasets and compare TAGSAM with 6 state-of-the-art baselines. On average, TAGSAM raises downstream models’ accuracy by 4.9% over the strongest baseline, and matches or even improves the running time.

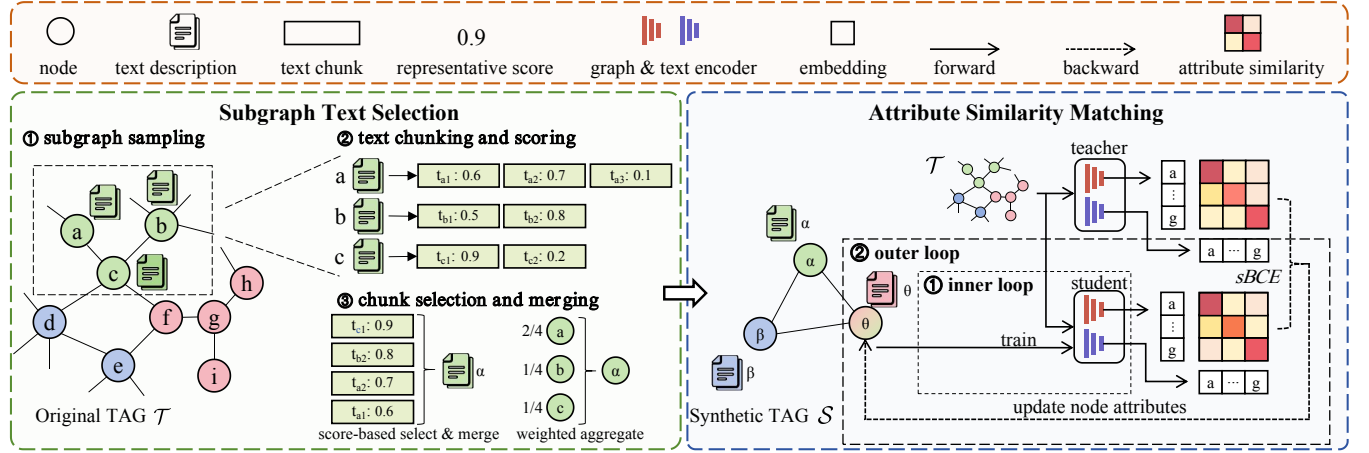


Figure 3: The overview of TAGSAM framework. Subgraph text selection (left) selects the representative chunks and merge them as a compressed text. Attribute similarity matching (right) compresses graph by minimizing the discrepancy between the attribute similarity matrices of the teacher and student models.

2 Preliminaries

Contrastive learning for TAG. Recent TAG models leverage contrastive learning to jointly learn graph and text embeddings. The goal is to maximize the similarity of positive pairs while minimizing that of negative pairs. For example, G2P2 [40] employs a GCN [16] and a transformer [31]-based text encoder to align representations of graph topology and node text. A common objective is InfoNCE [26], where the loss for a batch B of b nodes is defined as:

$$\ell_{con}(B) = -\frac{1}{b} \sum_{i=1}^b \left[\frac{\exp(s_{ii}/\tau)}{\sum_j \exp(s_{ij}/\tau)} + \frac{\exp(s_{ii}/\tau)}{\sum_j \exp(s_{ji}/\tau)} \right], \quad (1)$$

where s_{ij} is the cosine similarity between node i 's graph embedding and node j 's text embedding, and τ is the temperature coefficient.

Zero-shot classification for TAG. The contrastive learning leverages text as additional supervisory signal, which enables zero-shot classification. This allows the model to predict a node's class without requiring labeled examples during training [24]. This capability is highly valuable in TAGs because labels are expensive to obtain [40]. Detailed implementation can be found in Appendix C.

TAG Condensation Task. We introduce TAG condensation to reduce the high computational training cost. The goal is to compress a TAG $\mathcal{T} = (X, A, T)$ —with node features $X \in \mathbb{R}^{N \times d}$, adjacency $A \in \mathbb{R}^{N \times N}$, and node texts T —into a synthetic TAG $\mathcal{S} = (\hat{X}, \hat{A}, \hat{T})$ containing only $M \ll N$ nodes. The condensed representation, defined by $\hat{X} \in \mathbb{R}^{M \times d}$, $\hat{A} \in \mathbb{R}^{M \times M}$, and readable texts $\hat{T}$, is optimized to retain the information necessary for effective model training.

3 The TAGSAM Method

Overview. We propose a dataset condensation method dubbed as TAGSAM to compress the text-attributed graph dataset. The framework overview of TAGSAM is illustrated in Figure 3. Our method consists of two key condensation techniques: subgraph text selection and attribute similarity matching, which are used to compress

text and graph, respectively. In particular, we start by introducing the subgraph text selection, which compresses multiple text descriptions into a concise one by selecting representative chunks. Subsequently, we compress graph by stable attribute similarity matching, which avoids the high variance caused by MTT.

3.1 Subgraph Text Selection

Problem: unreadable text. Existing multi-modal dataset condensation methods optimize text directly in the embedding space via gradient descent. However, this approach has two critical limitations. First, the optimized embeddings are unreadable. Second, these embeddings are cannot be used as input for the tokenizer, rendering them incompatible with downstream language models and thus limiting their application. Therefore, we ask: *can we compress large-scale text descriptions into concise and readable texts?*

Solution: discrete text selection. To answer the research question, we propose subgraph text selection to compress several text descriptions into a concise one $\hat{T}_i$. The insight behind subgraph text selection is to select and merge representative text chunks from the original text descriptions, thereby eliminating the need to optimize continuous text embeddings. As depicted in Figure 3 (left), the subgraph text selection process involves three key steps:

- **Step ①: subgraph sampling.** We sample M subgraphs by randomly selecting a central node and performing a Breadth-First Search (BFS) until each subgraph reaches a size of g .
- **Step ②: text chunking and scoring.** We divide the texts within each subgraph $G_{\text{sub},i}$ into a set of chunks, where each chunk consists of a series of consecutive sentences. We denote the collection of all such chunks as D_i . Then, we compute a representative score for each text chunk within each D_i .
- **Step ③: chunk selection and merging.** From the set D_i , we select top k representative chunks and merge them to form a concise, compressed text $\hat{T}_i$. This process is repeated for each subgraph, resulting in M compressed text descriptions.

Subgraph text selection helps avoid the risk of blending unrelated texts by merging adjacent texts. Furthermore, compared to using the original texts directly as new text descriptions, our method provides the model with substantially better accuracy through enriched, non-redundant semantic information.

Selection criterion. After completing the aforementioned three condensation steps, we can preliminarily compress the text descriptions into a concise one. However, one issue remains unresolved: *what is the criterion for selecting text chunks?* To address this problem, we introduce Chunk-wise Mutual Information (CMI) to evaluate the representativeness of text chunks and select the chunk with the highest representative score for merging at each condensation step. The essence of the CMI is to measure the amount of shared information between different text chunks. Specifically, given two text chunks c_i and c_j within a subgraph $G_{\text{sub},o}$, their mutual information is calculated as follows:

$$\text{CMI}(c_i; c_j) = \log \frac{p(c_i, c_j)}{p(c_i) \cdot p(c_j)} = \log \frac{p(c_i|c_j)}{p(c_i)}. \quad (2)$$

After calculating the CMI between each pair of chunks, the next step is to select chunks to form a representative $\hat{T}_o$. Our design principle is to maximize the *information coverage* of D_o while minimizing *redundancy*. We define the representativeness score (RS) as follows:

$$\text{RS}(\hat{T}_o) = \underbrace{\sum_{\hat{c} \in \hat{T}_o} \sum_{c \in D_o} \text{CMI}(\hat{c}; c)}_{\text{Information Coverage}} - \underbrace{\sum_{\hat{c}_i, \hat{c}_j \in \hat{T}_o} \text{CMI}(\hat{c}_i; \hat{c}_j)}_{\text{Redundancy Penalty}}. \quad (3)$$

The first term maximizes the information correlation between $\hat{T}_o$ and D_o , while the second term penalizes redundancy in $\hat{T}_o$.

However, it is impractical to calculate the real probability distribution of chunks as shown in Equation 2. Therefore, we estimate the probability $p(c_i)$ of chunk c_i using a pre-trained causal language model ϕ . For conditional probabilities $p(c_i|c_j)$, we estimate it by calculating the probability of c_i when c_j is used as a prefix with ϕ . Finally, we can estimate the chunk-wise mutual information between c_i and c_j as follow:

$$\widehat{\text{CMI}}(c_i; c_j) = \frac{1}{2} \max \left(\log \frac{\phi(c_i|c_j)}{\phi(c_i)} + \log \frac{\phi(c_j|c_i)}{\phi(c_j)}, 0 \right). \quad (4)$$

We take the average because language model can not guarantee $\log \frac{\phi(c_i|c_j)}{\phi(c_i)} = \log \frac{\phi(c_j|c_i)}{\phi(c_j)}$. Zero truncation is applied to focus on chunk correlation and prevent misleading results from language model instability when taking logarithms.

Since the objective in Equation 3 is a combinatorial problem, solving it becomes increasingly costly as $|D_o|$ grows larger. To address this problem, we employ an approximate solution by selecting chunks sequentially using a greedy algorithm. In particular, we choose the next chunk from the D_o that provides the greatest improvement to the Equation 3, until we have selected k chunks at each condensation step.

Theoretical analysis. We also present a detailed theorem demonstrating that our greedy algorithm, which selects text chunks based on representative score, can approximate the theoretically optimal solution in complex combinatorial optimization.

THEOREM 1. *Given a cardinality constraint k and the natural logarithm base e , the greedy algorithm yields a solution $\hat{T}_o$ that approximates the optimal text selection T_o^* :*

$$\text{RS}(\hat{T}_o) \geq \left(1 - \frac{1}{e}\right) \text{RS}(T_o^*).$$

The proof is deferred to Appendix A. It confirms that greedy selection yields at least a constant fraction of the optimum.

3.2 Attribute Similarity Matching

After condensing texts, the next step is to condense the graph, including graph structure and node features.

Graph topology initialization. Through subgraph text selection, we derive M compressed text descriptions. First, to align each compressed description with a graph structure, we follow GCond [14] and employ an MLP trained on the original graph to predict links based on text embeddings between node pairs. This avoids nested optimization and is highly efficient, as the texts remain fixed. Next, to assign a corresponding node feature to each compressed text description, we perform a weighted aggregation of the original node features within each raw subgraph, as shown in step ③ of Figure 3. The weight for each node's features is determined by the proportion of text chunks selected from it.

Bi-level optimization for node feature. While generating node features through aforementioned weighted aggregation is a feasible approach, this simple operation struggles to achieve robust alignment with the compressed texts and the generated graph structure. To overcome this limitation, we adopt the well-established paradigm of bi-level optimization. This allows us to dynamically generate node features. We can detail this approach as follows:

- **Inner loop.** As shown in Figure 3, we optimize a model f called student model with randomly initialized parameter θ_s^0 by continuously reducing ℓ_{con} on the synthetic graph $\mathcal{S}$ with learning rate η as shown in Equation 5.

$$\theta_s^{t+1} \leftarrow \theta_s^t - \eta \nabla_{\theta} \ell_{\text{con}}(f_{\theta_s^t}(\mathcal{S})) \quad (5)$$

- **Outer loop.** Let $\Theta = \{\theta^0, \theta^1, \dots, \theta^*\}$ denote the set of parameters updated at each step. We aim to find a suitable $\hat{X}$, such that the set of student parameters Θ_s obtained from inner loop, aligns with the set of teacher parameters Θ_t obtained from training on the original graph $\mathcal{T}$. The alignment is measured by a matching loss function ℓ_{match} in a certain aspect. We denote it as follow:

$$\min_{\hat{X}} \mathbb{E}_{\theta_s^0 \sim P_{\theta_s^0}} \ell_{\text{match}}(\Theta_s, \Theta_t) \quad (6)$$

Problem: high variance in training trajectories. Current multi-modal dataset condensation methods often utilize the above bi-level optimization framework and employ Matching Training Trajectories (MTT) as their matching loss function ℓ_{match} . This approach operates by aligning the parameter update trajectories of teacher and student models across their respective training stages. However, when applying this strategy to TAG condensation, we observe that the compressed graph often fails to accurately represent the original. The core issue lies in the nature of the teacher models in the TAG scenario, which are pre-trained via contrastive learning. As demonstrated by [15], contrastive learning loss inherently

Table 1: Complexity analysis of matching trajectory training (MTT) and our attribute similarity matching (ASM).

Method	Teacher Model Training	Condensation			
		Forward ℓ_{con}	Update f_s	Loss ℓ_{match}	Update S
MTT	$O(Z(L_g E h_g + L_g B h_g^2 + L_t B h_t^2 + B^2))$	$O(q(L_g M h_g^2 + L_t M h_t^2))$	$O(q(L_g h_g^2 + L_t h_t^2))$	$O(L_g h_g^2 + L_t h_t^2)$	$O(M)$
ASM	$O(L_g E h_g + L_g B h_g^2 + L_t B h_t^2 + B^2)$	$O(q(L_g M h_g^2 + L_t M h_t^2))$	$O(q(L_g h_g^2 + L_t h_t^2))$	$O(L_g m h_g^2 + L_t m h_t^2 + m^2)$	$O(M)$

produces gradients with high variance. This is because the loss for any given sample is highly dependent on the other samples within the same batch, unlike in supervised learning where each example’s contribution is independent. The high variance in gradients causes unpredictable and unstable teacher trajectories. As a result, the trajectory matching task becomes a very difficult optimization problem. This instability, as shown by our observations in Figure 2, ultimately reduces the effectiveness of MTT for TAG condensation.

Solution: attribute similarity matching. To avoid the unstable optimization process associated with MTT, we design a condensation technique dubbed as *attribute similarity matching*. In particular, the product of graph embeddings $f^G(X)$ and text embeddings $f^T(T)$ between each pair of samples forms the attribute similarity matrices SM . We aim to minimize the discrepancy between the similarity matrices computed by the student model (i.e., SM_s) and the teacher model (i.e., SM_t) on the original samples, ensuring student model can capture the inter-modal relationships. Formally, we denote our matching loss function ℓ_{match} as follows:

$$SM_s = f_{\theta_s^G}^G(X) f_{\theta_s^T}^T(T)^\top, SM_t = f_{\theta_t^G}^G(X) f_{\theta_t^T}^T(T)^\top. \quad (7)$$

$$\ell_{match} = \frac{1}{2} (\ell_{sBCE}(SM_s, SM_t) + \ell_{sBCE}(SM_s^\top, SM_t^\top)), \quad (8)$$

where $sBCE$ means that applying a row-wise softmax to each similarity matrix before computing Binary Cross Entropy, giving smoother gradients and steadier alignment.

Theoretical analysis. Following, we provide a detailed explanation of why attribute similarity matching is more robust than matching training trajectories.

THEOREM 2. *In TAG contrastive learning, the optimal embeddings $f_G^*(X_i)$ and $f_T^*(T_i)$ are unique only up to a rotation matrix R followed by a scaling matrix S :*

$$f_G^*(X_i) = \frac{1}{\sqrt{p_X(i)}} (U_i^k S R)^\top, \quad (9)$$

$$f_T^*(T_i) = \frac{1}{\sqrt{p_T(i)}} (V_i^k \Sigma^k S^{-1} R)^\top, \quad (10)$$

where U^k, Σ^k, V^k are the rank- k factors of the normalized attribute joint distribution $\tilde{P}$.

Consequently, the logarithm of the optimal similarity s_{ij}^* between any node attribute X_i and text attribute T_j equals their pointwise mutual information:

$$\log s_{ij}^* = \log(f_G^*(X_i) f_T^*(T_j)^\top) = \text{PMI}(X_i, T_j). \quad (11)$$

The proof can be found in Appendix B. We observe two insights from Theorem 2 as follows:

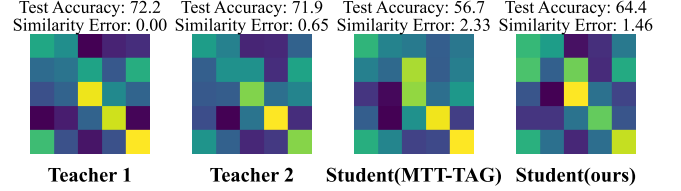


Figure 4: Visualization of attribute similarity matrices for teacher and student models. Similarity Error is the element-wise relative error with respect to Teacher 1.

- *Unstable training trajectory of MTT.* Equations 9 and 10 show that teacher embeddings are defined up to arbitrary rotation R and scaling S . Because MTT forces the student to follow the teacher’s full training trajectories, the search space becomes very large and optimization turns unstable. Meanwhile, random sampling in contrastive learning keeps these trajectories more fluctuated. As *Student (MTT-TAG)* in Figure 4 illustrates, the unstable training leads to lower accuracy.
- *Robust similarity matrix.* Equation 11 shows that the optimal element s_{ij}^* of this matrix is equal to the PMI between X_i and T_j , making them independent of the arbitrary matrices R and S in the graph and text encoders. Attribute similarity matching aligns the similarity matrices of the teacher and student models, thereby avoiding directly matching the graph and text encoders. Figure 4 shows that our student model achieves lower error and higher accuracy than the student model in MTT-TAG.

Complexity analysis. For an intuitive comparison, we present the detailed time complexity of matching training trajectory (MTT) and attribute similarity matching (ASM) in Table 1. Let E be the nodes and edges in the $\mathcal{T}$, and M the nodes in the $\mathcal{S}$. Z appears only in the teacher training process of MTT, representing the number of teachers. In the inner loop, the student model undergoes q epoch updates. Additionally, The model’s graph encoder has L_g layers with h_g hidden dimension, and the text encoder comprises L_t layers with h_t hidden dimension. B and m denote the batch sizes for teacher training and similarity matching, respectively.

According to Table 1, the overall time complexity of both methods is largely similar, except for the “Teacher Model Training” phase. MTT’s approach requires training Z (typically 15) separate teacher models for collecting varied training trajectories, making its cost prohibitive for large-scale graphs. In contrast, by matching stable similarity matrices, our ASM method needs only one teacher. This fundamental difference makes ASM a significantly more efficient and scalable solution for TAG condensation.

Table 2: Classification accuracy in percentage (%) for models trained on the condensed datasets (mean $\pm$ std). Best results are in bold. *Whole* refers to training on the original graph. Numbers below datasets indicate the original node count, while *Size* denotes the compressed node count. We calculate the accuracy increment by comparing against the best baseline in each setting.

Dataset	Size	Random	Herding	K-Center	GCond*	MTT-TAG	MTT-KNN	TAGSAM (Ours)	Whole
Cora (2,708)	50	50.0 $\pm$ 2.5	33.4 $\pm$ 2.7	47.3 $\pm$ 2.6	40.1 $\pm$ 3.9	64.4 $\pm$ 3.4	62.8 $\pm$ 1.6	73.8 $\pm$ 1.8 ($\uparrow$ 9.4)	78.7 $\pm$ 0.5
	100	68.1 $\pm$ 1.4	34.4 $\pm$ 2.1	61.9 $\pm$ 1.1	49.7 $\pm$ 3.6	73.8 $\pm$ 0.9	71.1 $\pm$ 1.7	75.2 $\pm$ 1.1 ($\uparrow$ 1.4)	
	200	71.8 $\pm$ 1.2	43.8 $\pm$ 0.9	68.3 $\pm$ 1.1	55.2 $\pm$ 2.9	76.1 $\pm$ 0.5	72.4 $\pm$ 1.2	76.6 $\pm$ 0.8 ($\uparrow$ 0.5)	
Photo (48,362)	100	41.4 $\pm$ 2.1	20.8 $\pm$ 1.1	40.8 $\pm$ 3.1	38.5 $\pm$ 4.3	51.7 $\pm$ 3.4	49.8 $\pm$ 2.7	56.4 $\pm$ 2.3 ($\uparrow$ 4.7)	63.3 $\pm$ 0.2
	200	50.4 $\pm$ 2.3	29.2 $\pm$ 2.1	45.3 $\pm$ 3.4	43.3 $\pm$ 3.9	56.2 $\pm$ 0.9	52.8 $\pm$ 0.8	60.8 $\pm$ 1.9 ($\uparrow$ 4.6)	
	500	59.1 $\pm$ 1.9	33.5 $\pm$ 0.6	53.6 $\pm$ 2.1	45.4 $\pm$ 4.2	59.1 $\pm$ 0.6	56.9 $\pm$ 0.9	63.3 $\pm$ 0.9 ($\uparrow$ 4.2)	
Computer (87,229)	100	32.4 $\pm$ 3.3	21.5 $\pm$ 3.8	42.4 $\pm$ 3.8	43.8 $\pm$ 4.9	48.0 $\pm$ 1.7	37.3 $\pm$ 3.1	62.1 $\pm$ 0.6 ($\uparrow$ 14.1)	72.2 $\pm$ 0.3
	200	46.4 $\pm$ 3.0	30.4 $\pm$ 1.7	45.4 $\pm$ 2.1	48.4 $\pm$ 2.2	56.7 $\pm$ 1.6	56.7 $\pm$ 2.9	64.4 $\pm$ 0.3 ($\uparrow$ 7.7)	
	500	49.8 $\pm$ 2.5	35.3 $\pm$ 2.5	49.0 $\pm$ 3.1	50.1 $\pm$ 2.0	61.6 $\pm$ 0.9	59.8 $\pm$ 1.2	70.6 $\pm$ 0.4 ($\uparrow$ 9.0)	
Arxiv (169,343)	100	46.3 $\pm$ 1.1	28.3 $\pm$ 1.4	45.2 $\pm$ 1.1	27.7 $\pm$ 3.1	22.7 $\pm$ 0.9	20.3 $\pm$ 3.3	49.6 $\pm$ 1.1 ($\uparrow$ 3.3)	55.3 $\pm$ 0.3
	200	47.5 $\pm$ 1.2	29.7 $\pm$ 0.6	47.0 $\pm$ 0.7	35.8 $\pm$ 3.4	30.5 $\pm$ 1.6	28.8 $\pm$ 3.4	51.9 $\pm$ 1.5 ($\uparrow$ 4.4)	
	500	52.7 $\pm$ 0.5	32.7 $\pm$ 0.4	52.8 $\pm$ 0.5	42.3 $\pm$ 1.9	49.5 $\pm$ 0.5	48.1 $\pm$ 2.3	55.4 $\pm$ 0.6 ($\uparrow$ 2.6)	
Products (2,449,029)	500	52.7 $\pm$ 2.3	17.8 $\pm$ 1.4	26.3 $\pm$ 2.4	39.0 $\pm$ 1.8	48.4 $\pm$ 1.6	46.2 $\pm$ 3.2	55.4 $\pm$ 1.0 ($\uparrow$ 2.7)	57.6 $\pm$ 0.1
	1000	53.8 $\pm$ 1.8	25.8 $\pm$ 0.7	29.9 $\pm$ 2.3	45.8 $\pm$ 2.0	50.0 $\pm$ 2.4	47.6 $\pm$ 2.1	55.8 $\pm$ 0.6 ($\uparrow$ 2.0)	
	2000	54.8 $\pm$ 2.0	33.7 $\pm$ 0.7	38.5 $\pm$ 2.4	47.5 $\pm$ 1.5	53.3 $\pm$ 1.7	49.5 $\pm$ 2.3	57.3 $\pm$ 0.2 ($\uparrow$ 2.5)	

4 Experimental Evaluations

4.1 Experiment Settings

Datasets. We benchmark TAGSAM on 5 text-attributed graphs. *Cora* [29] and *ArXiv* [32] are citation networks whose nodes are papers described by titles and abstracts, with edges representing citations. *Photo*, *Computers* [30], and *Products* [11] are Amazon co-purchase graphs; each node is a product whose text attribute is its most-voted user review, and edges indicate co-purchase interactions. These datasets vary widely in scale, from 2.7 K to 2.4 M nodes. More details about datasets are provided in Appendix D.

Baselines. We compare TAGSAM with 6 representative baselines.

- **Selective methods.** (a) *Random* selects samples randomly. (b) *Herding* [39] greedily selects samples in order to minimize the distance between the center of the coreset and the center of the full dataset in feature space. (c) *K-Center* [7] aims to select coreset samples that maximize their dispersion in the feature space.
- **Generative methods.** (a) *GCond** adapts GCond [14] for text-attributed graphs by extending its original graph-only gradient matching to a joint graph-text objective. (b) *MTT-TAG* adapts MTT-VL [41], an image-text multi-modal condensation method, by replacing its image encoder with a graph encoder. (c) *MTT-KNN* further compresses text into optimized embeddings, then employs a K-Nearest Neighbors (KNN) mapping, a variant approach proposed by MTT-VL, to retrieve the closest original text.

Evaluation protocol. Models are then trained on condensed graph and evaluated on the original test split. Following the five-way zero-shot classification setting of G2P2 [40], we report the classification accuracy over 5 independent runs. By default, GCN [16] and BERT [5] with a projector serve as the graph and text encoders. More hyperparameter settings can be found in Appendix E.

Table 3: Cross-architecture classification accuracy for models trained on the condensed Computer which is compressed using a combination of GCN and Bert.

Method	Text Encoder	Graph Encoder			
		GCN	MLP	SAGE	Cheby
Random	BERT	43.6	39.0	35.2	33.4
MTT-TAG		62.1	38.0	34.8	35.0
TAGSAM		64.4	46.9	45.5	46.2
Random	RoBERTa	28.5	25.2	25.0	23.6
MTT-TAG		39.3	27.5	29.7	28.3
TAGSAM		47.7	35.4	35.7	36.3

4.2 Main Results

Classification performance. To assess the quality of the compressed TAGs, Table 2 reports the classification accuracy of all baselines and our TAGSAM. We make the following observations:

Selective methods like Herding, and K-Center perform relatively poorly, confirming that simple heuristic sampling without learning cannot effectively preserve information from original TAG. Random sampling sometimes outperforms most other methods, especially on datasets with many classes (e.g. ArXiv and Products). This phenomenon has also been observed in image-text dataset condensation [41]. We hypothesize this is because without class guidance, their selection heuristics can introduce significant sampling bias by high-density features that may not correlate with class diversity. Random sampling, in contrast, is less affected as it is agnostic to these potentially misleading data structures.

GCond* performs poorly because its gradient-based matching strategy is designed to match the gradients of samples with the same class label. However, in TAG condensation, where no class

Table 4: Link prediction AUC for models trained on the condensed datasets (mean±std). Best results are in bold.

Dataset	Random	MTT-TAG	TAGSAM
Cora	75.85 ± 1.32	78.26 ± 1.34	82.15 ± 0.97
Photo	84.34 ± 1.43	86.44 ± 1.41	87.69 ± 1.20
Computer	87.62 ± 2.15	88.33 ± 1.74	91.38 ± 1.83
Arxiv	81.93 ± 2.04	86.35 ± 1.22	87.37 ± 1.57
Products	84.21 ± 2.20	84.62 ± 2.39	86.06 ± 2.05

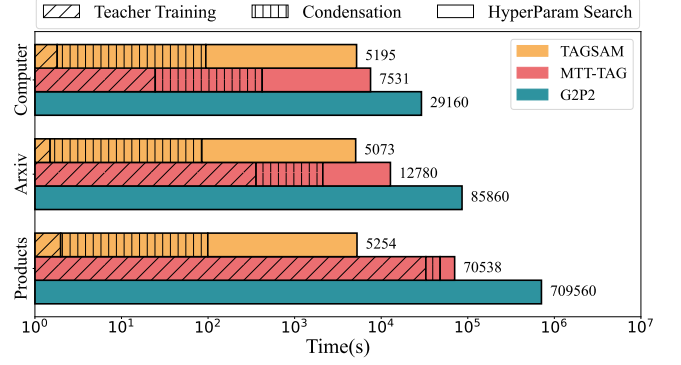
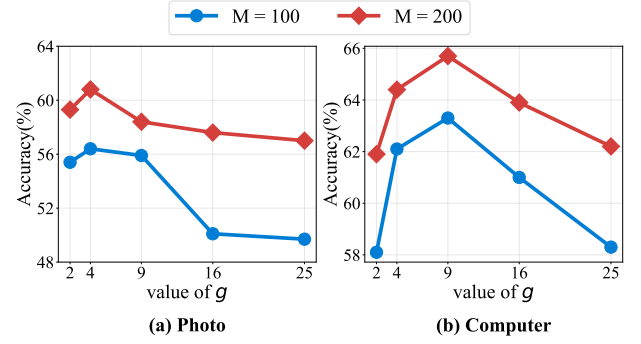
label can be used, this approach is ineffective because matching gradients between different class samples will lead to instability. Furthermore, such conventional graph condensation methods are designed to compress graph modality only, which means they are limited by their inability to compress text modality.

Existing multi-modal condensation methods like MTT-KNN and MTT-TAG generally perform best on average. Notably, the accuracy of MTT-KNN is lower than MTT-TAG. This is because MTT-KNN compresses texts into embedding vectors, for which it has to find the nearest neighbor original text descriptions. Such an operation disrupts the alignment between the compressed node features and texts produced by the learning process. Despite their higher performance, they are also limited by their inability to compress discrete text descriptions and suffer from unstable matching objectives. Our proposed TAGSAM overcomes these limitations, achieving superior performance across all datasets and condensation sizes. It boosts accuracy by up to 14.1% over the strongest baseline MTT-TAG. For larger condensation sizes, TAGSAM’s accuracy, although demonstrating more variability compared to training on the Whole dataset, reaches an average value highly comparable to it.

Cross-architecture generalization. Table 3 evaluates cross architecture generalization by condensing the Computer dataset to 200 nodes. We compare our method, TAGSAM, with top baselines: Random for selective condensation and MTT-TAG for generative condensation. Specifically, we use a GCN+BERT architecture to condense the dataset for TAGSAM and MTT-TAG. Subsequently, these small, condensed graphs are used to train various other model combinations, including graph encoders (GCN, MLP, SAGE [9], Cheby [4]) and text encoders (BERT [5], Roberta [21]).

The results in Table 3 suggest that all methods experience accuracy degradation. The degradation also occurs in Random, demonstrating that there is significant accuracy loss caused by the encoder’s lower compatibility. For the MTT-TAG and TAGSAM, the degradation is additionally caused by the use of different architectures during the condensation and testing phases. However, TAGSAM consistently achieves the highest accuracy due to our effective and stable condensation. Compared to random sampling, TAGSAM shows a substantial average improvement of 13.1%.

Link prediction performance. We evaluate link prediction by training an MLP on features extracted from the graph encoder. As shown in Table 4, our TAGSAM outperforms the top selective (Random) and generative (MTT-TAG) baselines, achieving significant AUC improvements of 4.1% and 2.1% respectively. This performance suggests our method more effectively retains structural information

**Figure 5: Running time (in log scale) for hyperparameter search of downstream model. G2P2 trains directly on the original graph, while TAGSAM and MTT-TAG first compress the graph and then train models on the condensed graph.****Figure 6: The impact of subgraph size for subgraph text selection. M denotes the number of compressed texts/nodes.**

from the original graph, thereby enhancing the models trained on the resulting condensed graph.

Efficiency study. We study the impact of TAG condensation on accelerating hyperparameter searches for downstream TAG models, and compare running time of TAGSAM, MTT-TAG and G2P2 (direct training). Specifically, we use 108 hyperparameter settings: optimizers {SGD, Adam, AdamW}; graph encoder depths {2, 3, 4}; projector depths {1, 2, 3}; and embedding sizes {64, 128, 192, 256}. We also decompose the running time of MTT-TAG and TAGSAM into *teacher training*, *condensation*, and *hyperparameter search*.

The results in Figure 5 show that the larger the dataset, the greater the time savings from condensation. Although condensation incurs a one-time cost, it significantly reduces the time for each subsequent hyperparameter trial, meaning its advantage grows with the number of searches. For larger datasets, teacher model training dominates total runtime. TAGSAM requires training only a single teacher due to its attribute similarity consistency (Figure 4), eliminating the need for additional teachers, unlike MTT-TAG which must train multiple teachers to reflect diverse training trajectories.

Hyperparameter sensitivity. We study the hyperparameter sensitivity experiments with respect to the subgraph size g in subgraph

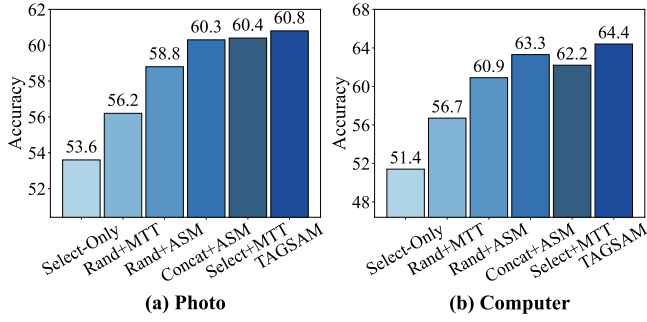


Figure 7: The ablation study for different variants. The y-axis is anchored at the accuracy of the baseline *Random*.

text selection on the Photo and Computer datasets. The results are shown in Figure 6. A larger g means more information is integrated, but it can also lead to more information confusion.

We validate the impact of g on compression performance through experiments on the Photo and Computer datasets. We find that setting $g = 4$ and $g = 9$ yields the best performance for the downstream models. Additionally, we observe a sharp performance drop when $g \geq 16$. We attribute this decline to the fact that such a large g involves selecting many 2-hop neighbors, which introduces semantically disparate texts, making it difficult to optimize the alignment of the corresponding node features for the newly compressed text.

4.3 Ablation Study

As reported in Figure 7, we experiment under the setting of synthetic dataset size $M = 200$, with 5 variants as follows:

- *Select-Only* conducts subgraph text selection to compress TAG.
- *Rand+MTT* randomly selects texts and compresses graph via matching training trajectories.
- *Rand+ASM* randomly selects texts and compresses graph via our attribute similarity matching.
- *Concat+ASM* samples subgraphs and directly concatenates all the texts within each subgraph to form a new text, then compresses graph via attribute similarity matching.
- *Select+MTT* compresses the text and graph by our subgraph text selection and matching training trajectories respectively.

The efficacy of text selection. Comparing “Select-Only” to the “Random” baseline confirms that text selection enriches the text modality for improved training. The comparison between “Rand+ASM” and “Concat+ASM” also reveals that text concatenation can enhance performance. However, this naive strategy is outperformed by “TAGSAM” as it introduces significant redundancy and noise, limiting its performance. In contrast, our method’s precise selection mechanism yields superior performance, improving the ASM framework by 2.8%. This improvement is more pronounced for the MTT strategy, reaching 4.9%. These results demonstrate the significance and robustness of our subgraph text selection approach.

Attribute vs. Trajectory Matching. As illustrated in Figure 7, a clear performance advantage of our proposed Attribute Similarity Matching (ASM) is evident. A direct comparison between “Rand+MTT” and “Rand+ASM”, as well as between “Select+MTT”

and our “TAGSAM” model, consistently demonstrates that ASM significantly and reliably outperforms the Matching Training Trajectories (MTT) baseline across different text compression.

5 Related Work

Selection-based condensation. Coreset selection is a critical technique that compresses large datasets into smaller subsets by selecting important samples. There are various pioneer literature reduces the computational burden of training on large datasets while maintaining model performance [2, 7, 39]. Main approaches are random sampling, Herding, which aligns subset and dataset means [39], and K-center, which minimizes the worst-case point-to-center distance [7]. All simply select existing data rather than synthesizing new samples; consequently, their performance is often sub-optimal.

Graph dataset condensation. Graph condensation aims at efficient GNN training [13, 19, 23] by condensing a large graph into a small synthetic one without significant performance loss. Recently, numerous dataset condensation methods are developed for compressing graph datasets [6, 8, 33, 37, 42, 44]. GCond [14] matches training gradients between the original and condensed graphs. GCDM [20] forces the spatial distribution of receptive fields in the condensed graph to mirrors that of the original dataset, i.e., distribution matching. However, these existing methods cannot handle the multi-modal setting of TAG condensation and rely on class-wise matching, making them heavily label-dependent.

Multi-modal dataset condensation. Unlike uni-modal methods, multi-modal approaches compress multiple modalities simultaneously. MTT-VL [41] first condense large-scale image-text datasets by matching training trajectories (MTT) into smaller dataset while maintaining effective training. LoRS [43] introduces a plug-and-play method that compresses image-text datasets by providing ground truth similarity between synthetic samples and reducing complexity with low-rank approximations. AVDD [17] condenses audio-visual data by aligning modalities through implicit cross-matching. However, these methods cannot compress discrete text modalities. and such MTT-based methods often prove unstable in contrastive learning settings, resulting in sub-optimal performance.

6 Conclusion

In this work, we introduce TAGSAM, a novel dataset condensation method for Text-Attributed Graphs (TAGs) that addresses the issue that existing methods cannot effectively compress TAGs. Our method utilizes subgraph text selection to compress multiple texts into a representative and discrete one and attribute similarity matching to assign robust synthetic attributes. Experiments show that models trained on graphs condensed by TAGSAM to just 1% of their original size still maintain competitive performance, thus making efficient pre-training on large-scale TAGs feasible.

Acknowledgments

This work was supported by the National Natural Science Foundation of China (No. 62472327, 62441229), the Fundamental Research Funds for the Central Universities (No. 2042025kf0040), and the Key R&D Program of Hubei Province (No. 2023BAB077).

References

- [1] William Brannon, Wonjune Kang, Suyash Fulay, Hang Jiang, Brandon Roy, Deb Roy, and Jad Kabbara. 2024. ConGraT: Self-Supervised Contrastive Pretraining for Joint Graph and Text Embeddings. In *Proceedings of the Workshop on Graph-based Methods for Natural Language Processing (TextGraphs)*.
- [2] Vladimir Braverman, Vincent Cohen-Addad, H-C Shao-feng Jiang, Robert Krauthgamer, Chris Schwiegelshohn, Mads Bech Tofttrup, and Xuan Wu. 2022. The power of uniform sampling for coresets. In *Proceedings of the Annual IEEE Symposium on Foundations of Computer Science (FOCS)*.
- [3] Jonathan Chang and David Blei. 2009. Relational Topic Models for Document Networks. In *Proceedings of the International Conference on Artificial Intelligence and Statistics (AISTATS)*.
- [4] Michaël Defferrard, Xavier Bresson, and Pierre Vandergheynst. 2016. Convolutional neural networks on graphs with fast localized spectral filtering. In *Proceedings of the Conference on Neural Information Processing Systems (NeurIPS)*.
- [5] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*.
- [6] Junfeng Fang, Xinglin Li, Yongduo Sui, Yuan Gao, Guibin Zhang, Kun Wang, Xiang Wang, and Xiangnan He. 2024. EXGC: Bridging Efficiency and Explainability in Graph Condensation. In *Proceedings of the Web Conference (WWW)*.
- [7] Reza Zanjirani Farahani and Masoud Hekmatfar. 2009. *Facility location: concepts, models, algorithms and case studies*. Springer Science & Business Media.
- [8] Xinyi Gao, Guanhua Ye, Tong Chen, Wentao Zhang, Junliang Yu, and Hongzhi Yin. 2025. Rethinking and Accelerating Graph Condensation: A Training-Free Approach with Class Partition. In *Proceedings of the Web Conference (WWW)*.
- [9] Will Hamilton, Zhitao Ying, and Jure Leskovec. 2017. Inductive representation learning on large graphs. In *Proceedings of the Conference on Neural Information Processing Systems (NeurIPS)*.
- [10] Sarel Har-Peled, Dan Roth, and Dav A Zimak. 2006. Maximum margin coresets for active and noise tolerant learning. In *Proceedings of the International Joint Conference on Artificial Intelligence (IJCAI)*.
- [11] Weihua Hu, Matthias Fey, Marinka Zitnik, Yuxiao Dong, Hongyu Ren, Bowen Liu, Michele Catasta, and Jure Leskovec. 2020. Open graph benchmark: datasets for machine learning on graphs. In *Proceedings of the Conference on Neural Information Processing Systems (NeurIPS)*.
- [12] Qiang Huang, Xiao Yan, Xin Wang, Susie Xi Rao, Zhichao Han, Fangcheng Fu, Wentao Zhang, and Jiawei Jiang. 2024. Retrofitting temporal graph neural networks with transformer. *arXiv preprint arXiv:2409.05477* (2024).
- [13] Song Jin, Xiantao Cai, and Jiawei Jiang. 2025. EDGaE: Efficient Distributed Graph Neural Network Training System at the Edge. In *Proceedings of the Advanced Intelligent Computing Technology and Applications (ICIT)*.
- [14] Wei Jin, Lingxiao Zhao, Shichang Zhang, Yozen Liu, Jiliang Tang, and Neil Shah. 2021. Graph condensation for graph neural networks. In *Proceedings of the International Conference on Learning Representations (ICLR)*.
- [15] Siddharth Joshi, Jiayi Ni, and Baharan Mirzasoleiman. 2024. Dataset distillation via knowledge distillation: towards efficient self-supervised pre-training of deep networks. In *Proceedings of the International Conference on Learning Representations (ICLR)*.
- [16] Thomas N Kipf and Max Welling. 2016. Semi-supervised classification with graph convolutional networks. In *Proceedings of the International Conference on Learning Representations (ICLR)*.
- [17] Saksham Singh Kushwaha, Siva Sai Nagender Vasireddy, Kai Wang, and Yapeng Tian. 2024. Audio-visual dataset distillation. *Transactions on Machine Learning Research (TMLR)* (2024).
- [18] Yichuan Li, Kaize Ding, and Kyumin Lee. 2023. GRENADE: graph-centric language model for self-supervised representation learning on text-attributed graphs. In *Findings of the Association for Computational Linguistics: EMNLP*.
- [19] Yuxuan Liang, Wentao Zhang, Zeang Sheng, Ling Yang, Quanqing Xu, Jiawei Jiang, Yunhai Tong, and Bin Cui. 2025. Towards Scalable and Deep Graph Neural Networks via Noise Masking. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*.
- [20] Mengyang Liu, Shanchuan Li, Xinshi Chen, and Le Song. 2022. Graph condensation via receptive field distribution matching. *arXiv preprint arXiv:2206.13697* (2022).
- [21] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. RoBERTa: a robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692* (2019).
- [22] Yunhui Liu, Qizhuo Xie, Jinwei Shi, Jiaxu Shen, and Tiek He. 2025. Multi-Scale Heterogeneous Text-Attributed Graph Datasets From Diverse Domains. In *Proceedings of the Web Conference (WWW)*.
- [23] Lu Ma, Zeang Sheng, Xunkai Li, Xinyi Gao, Zhezheng Hao, Ling Yang, Xiaonan Nie, Jiawei Jiang, Wentao Zhang, and Bin Cui. 2025. Acceleration Algorithms in GNNs: A Survey. *Transactions on Knowledge and Data Engineering (TKDE)* (2025).
- [24] Bishwas Mandal, Sarthak Khanal, and Doina Caragea. 2024. Contrastive Learning for Multimodal Classification of Crisis related Tweets. In *Proceedings of the Web Conference (WWW)*.
- [25] G. L. Nemhauser, L. A. Wolsey, and M. L. Fisher. 1978. An analysis of approximations for maximizing submodular set functions—I. *Math. Program.* (1978).
- [26] Aaron van den Oord, Yazhe Li, and Oriol Vinyals. 2018. Representation learning with contrastive predictive coding. *arXiv preprint arXiv:1807.03748* (2018).
- [27] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. 2019. Language models are unsupervised multitask learners. *OpenAI Blog* (2019). <https://api.semanticscholar.org/CorpusID:160025533>
- [28] Sebastian Ruder. 2016. An overview of gradient descent optimization algorithms. *arXiv preprint arXiv:1609.04747* (2016).
- [29] Prithviraj Sen, Galileo Namata, Mustafa Bilgic, Lise Getoor, Brian Galligher, and Tina Eliassi-Rad. 2008. Collective classification in network data. *AI Magazine* (2008).
- [30] Oleksandr Shchur, Maximilian Mumme, Aleksandar Bojchevski, and Stephan Günnemann. 2018. Pitfalls of graph neural network evaluation. *arXiv preprint arXiv:1811.05868* (2018).
- [31] A Vaswani. 2017. Attention is all you need. In *Proceedings of the Conference on Neural Information Processing Systems (NeurIPS)*.
- [32] Kuansan Wang, Zhihong Shen, Chiyan Huang, Chieh-Han Wu, Yuxiao Dong, and Anshul Kanakia. 2020. Microsoft academic graph: When experts are not enough. *Quantitative Science Studies* (2020).
- [33] Lin Wang, Wenqi Fan, Jiatong Li, Yao Ma, and Qing Li. 2024. Fast Graph Condensation with Structure-based Neural Tangent Kernel. In *Proceedings of the Web Conference (WWW)*.
- [34] Tongzhou Wang, Jun-Yan Zhu, Antonio Torralba, and Alexei A Efros. 2018. Dataset distillation. *arXiv preprint arXiv:1811.10959* (2018).
- [35] Xin Wang, Jiawei Jiang, Xiao Yan, and Qiang Huang. 2025. TESA: A Trajectory and Semantic-aware Dynamic Heterogeneous Graph Neural Network. In *Proceedings of the Web Conference (WWW)*.
- [36] Yuxiang Wang, Xiao Yan, Chuang Hu, Quanqing Xu, Chuanhui Yang, Fangcheng Fu, Wentao Zhang, Hao Wang, Bo Du, and Jiawei Jiang. 2024. Generative and Contrastive Paradigms Are Complementary for Graph Self-Supervised Learning. In *International Conference on Data Engineering (ICDE)*.
- [37] Yuxiang Wang, Xiao Yan, Shiyu Jin, Hao Huang, Quanqing Xu, Qingchen Zhang, Bo Du, and Jiawei Jiang. 2024. Self-Supervised Learning for Graph Dataset Condensation. In *Proceedings of the Conference on Knowledge Discovery and Data Mining (KDD)*.
- [38] Yuxiang Wang, Xiao Yan, Shiyu Jin, Quanqing Xu, Chuang Hu, Yuanyuan Zhu, Bo Du, Jia Wu, and Jiawei Jiang. 2025. Exploiting Text Semantics for Few and Zero Shot Node Classification on Text-attributed Graph. In *Proceedings of the International Joint Conference on Artificial Intelligence (IJCAI)*.
- [39] Max Welling. 2009. Herding dynamical weights to learn. In *Proceedings of the International Conference on Machine Learning (ICML)*.
- [40] Zhihao Wen and Yuan Fang. 2023. Augmenting low-resource text classification with graph-grounded pre-training and prompting. In *Proceedings of the International Conference on Research and Development in Information Retrieval (SIGIR)*.
- [41] Xindi Wu, Zhiwei Deng, and Olga Russakovsky. 2023. Vision-Language Dataset Distillation. *Transactions on Machine Learning Research (TMLR)* (2023).
- [42] Zhenbang Xiao, Yu Wang, Shunyu Liu, Bingde Hu, Huiqiong Wang, Mingli Song, and Tongya Zheng. 2025. Disentangled Condensation for Large-scale Graphs. In *Proceedings of the Web Conference (WWW)*.
- [43] Yue Xu, Zhilin Lin, Yusong Qiu, Cewu Lu, and Yong-Lu Li. 2024. Low-rank similarity mining for multimodal dataset distillation. In *Proceedings of the International Conference on Machine Learning (ICML)*.
- [44] Hongzhi Yin, Xinyi Gao, Junliang Yu, Ruihong Qiu, Tong Chen, Quoc Viet Hung Nguyen, and Zi Huang. 2025. Graph Condensation: Foundations, Methods and Prospects. In *Proceedings of the Web Conference (WWW)*.
- [45] Qi Zhang, Yifei Wang, and Yisen Wang. 2023. On the generalization of multimodal contrastive learning. In *Proceedings of the International Conference on Machine Learning (ICML)*.
- [46] Bo Zhao and Hakan Bilen. 2023. Dataset condensation with distribution matching. In *Proceedings of the Winter Conference on Applications of Computer Vision (WACV)*.
- [47] Bo Zhao, Konda Reddy Mopuri, and Hakan Bilen. 2020. Dataset condensation with gradient matching. In *Proceedings of the International Conference on Learning Representations (ICLR)*.
- [48] Yun Zhu, Haizhou Shi, Xiaotang Wang, Yongchao Liu, Yaoke Wang, Boci Peng, Chuntao Hong, and Siliang Tang. 2024. Graphclip: enhancing transferability in graph foundation models for text-attributed graphs. In *Proceedings of the Web Conference (WWW)*.

A Proof of Theorem 1

PROOF. Firstly, consider the impact of including c_i in $\hat{T}_o$, where $\hat{T}_o \subseteq D_o$ and $c_i \in D_o \setminus \hat{T}_o$. The contribution of c_i is calculated as:

$$\sum_{c_j \in D_o} \text{CMI}(c_i, c_j) - \sum_{c_j \in \hat{T}_o} \text{CMI}(c_i, c_j) = \sum_{c_j \in D_o \setminus \hat{T}_o} \text{CMI}(c_i, c_j).$$

As the estimated CMI values are non-negative, this term is clearly positive. This demonstrates that RS is monotonic, implying that

$$\text{RS}(\hat{T}_o \cup \{c_i\}) \geq \text{RS}(\hat{T}_o).$$

Secondly, let $A \subseteq B \subseteq D_o$ and $c_i \in D_o \setminus (A \cup B)$. The improvement of adding c_i to both A and B is characterized by the same information coverage gain $\sum_{c_j \in D_o} \text{CMI}(c_i, c_j)$. However, the redundancy penalty is larger for B because the difference

$$\sum_{c_j \in B \setminus A} \text{CMI}(c_i, c_j) = \sum_{c_j \in B} \text{CMI}(c_i, c_j) - \sum_{c_j \in A} \text{CMI}(c_i, c_j)$$

is positive. Therefore, submodularity is satisfied:

$$\text{RS}(A \cup \{c_i\}) - \text{RS}(A) \geq \text{RS}(B \cup \{c_i\}) - \text{RS}(B).$$

Finally, according to Nemhauser's theorem [25], as $\text{RS}(\hat{T}_o)$ is monotonic and submodular, applying a greedy algorithm to maximize $\text{RS}(\hat{T}_o)$ under a cardinality constraint k guarantees that

$$\text{RS}(\hat{T}_o) \geq \left(1 - \frac{1}{e}\right) \text{RS}(T_o^*).$$

□

B Proof of Theorem 2

PROOF. Following Zhang et al. [45], we adopt the TAG contrastive loss for ease of analysis:

$$\begin{aligned} \mathcal{L}_{\text{con}} = & -2 \mathbb{E}_{X_i, T_i} [f_G(X_i)^\top f_T(T_i)] \\ & + \mathbb{E}_{X_i, T_j} [(f_G(X_i)^\top f_T(T_j))^2], \end{aligned} \quad (12)$$

where (X_i, T_i) denotes a positive graph-text pair and (X_i, T_j) with $j \neq i$ denotes an independently sampled negative pair.

Let $P \in \mathbb{R}^{N \times N}$ be the joint-probability matrix whose (i, j) -entry is:

$$P_{ij} = p_{G,T}(i, j).$$

Define the marginal probabilities:

$$p_G(i) = \sum_j p_{G,T}(i, j), \quad p_T(j) = \sum_i p_{G,T}(i, j),$$

and form the *normalized* co-occurrence matrix:

$$\tilde{P}_{ij} = \frac{p_{G,T}(i, j)}{\sqrt{p_G(i) p_T(j)}}.$$

Consider the asymmetric low-rank factorization loss:

$$\mathcal{L}_{\text{AMF}}(F_G, F_T) = \|\tilde{P} - F_G F_T^\top\|_F^2, \quad (13)$$

where $F_G \in \mathbb{R}^{N \times k}$ and $F_T \in \mathbb{R}^{N \times k}$ with $k \ll N$.

Let the i -th row of F_G and the j -th row of F_T represent the encoded features of samples X_i and T_j , respectively, in the following form:

$$F_G(i) = \sqrt{p_G(i)} f_G(X_i)^\top, \quad (14a)$$

$$F_T(j) = \sqrt{p_T(j)} f_T(T_j)^\top. \quad (14b)$$

Substituting Equation 14 into Equation 13 and expanding the Frobenius norm yields:

$$\begin{aligned} \mathcal{L}_{\text{AMF}} &= \sum_{i,j} \left(\tilde{P}_{ij} - \sqrt{p_G(i)} f_G(X_i)^\top \sqrt{p_T(j)} f_T(T_j) \right)^2 \\ &= \sum_{i,j} \frac{p_{G,T}(i, j)^2}{p_G(i) p_T(j)} - 2 \mathbb{E}_{X_i, T_j} [f_G(X_i)^\top f_T(T_j)] \\ &\quad + \mathbb{E}_{X_i, T_j} [(f_G(X_i)^\top f_T(T_j))^2]. \end{aligned}$$

The first summation depends only on the data distribution, so denote it by the constant C . Recognizing the remaining two terms as the contrastive loss $\mathcal{L}_{\text{con}}$ in Eq. (12), we obtain:

$$\mathcal{L}_{\text{AMF}}(F_G, F_T) = \mathcal{L}_{\text{con}}(f_G, f_T) + C.$$

Because the additive constant C is independent of the embeddings, minimizing the matrix factorization loss is equivalent to minimizing the contrastive loss.

By the classical Eckart–Young theorem, the best rank- k approximation of the normalized co-occurrence matrix $\tilde{P}$ is its truncated SVD $U^k \Sigma^k V^{k\top}$. Every pair of factor matrices that attains this optimum can therefore be written as:

$$F_G^* = U^k D R, \quad F_T^* = V^k \Sigma^k D^{-1} R,$$

where $R \in \mathbb{R}^{k \times k}$ is an arbitrary unitary matrix and D is an arbitrary invertible diagonal matrix. Mapping the rows of F_G^* and F_T^* back to the encoder outputs yields:

$$f_G^*(X_i) = \frac{1}{\sqrt{p_G(i)}} (U_i^k D R)^\top,$$

$$f_T^*(T_j) = \frac{1}{\sqrt{p_T(j)}} (V_j^k \Sigma^k D^{-1} R)^\top.$$

Hence the optimal graph and text embeddings are uniquely determined up to a rotation R and scaling D , as claimed in Theorem 2.

Finally, taking the inner product of the optimal embeddings and applying the logarithm gives:

$$\begin{aligned} \log s_{ij}^* &= \log(f_G^*(X_i)^\top f_T^*(T_j)) \\ &= \log \frac{p_{G,T}(i, j)}{p_G(i) p_T(j)} = \log \frac{P(X_i, T_j)}{P(X_i) P(T_j)} \\ &= \text{PMI}(X_i, T_j). \end{aligned}$$

Thus the optimal similarity is exactly the point-wise mutual information between X_i and T_j , which completes the proof of Theorem 2. □

Table 5: Statistics of Text-Attributed Graph datasets.

Dataset	# Node	# Edge	# Avg. Deg	# Class
Cora	2,708	5,429	4.01	7
Photo	48,362	500,928	20.72	12
Computers	87,229	721,081	16.53	10
ArXiv	169,343	1,166,243	13.77	40
Products	2,449,029	61,859,140	50.52	47

C Zero-shot Classification

Following the G2P2 [36, 40], in this paper, zero-shot classification aligns graph and text representations within a unified semantic space using contrastive learning. The architecture employs a graph encoder to generate an embedding for each node and a text encoder to transform text prompts describing potential classes—including those unseen during training—into corresponding class embeddings. For inference, the model classifies a node by calculating the cosine similarity between its embedding and all candidate class embeddings. The node is then assigned to the class with the highest similarity score, enabling effective classification into novel categories without any specific training examples for them.

D Dataset Details

Table 5 brings together five widely adopted TAG benchmarks that span orders of magnitude in scale and density while covering both academic and e-commerce domains, ensuring that our evaluation stresses every component of the condensation pipeline.

Cora [29]. This classic citation network contains 2,708 papers linked by 5,429 citation edges. Each node is accompanied by the raw abstract—typically 60 – 140 words—that spells out the problem statement, core methodology, and main results.

Amazon Photo and Amazon Computers [30]. Both graphs are extracted from the Amazon co-purchase network but focus on different product verticals. Amazon Photo (48 k nodes, 500 k edges) centers on cameras, lenses, lighting and accessories, whereas Amazon Computers (87 k nodes, 721 k edges) covers laptops, components and peripherals. For every product node we concatenate the seller’s catalogue title, the marketing blurb, and the first three customer-review snippets, yielding on average 35 – 50 tokens of

plain-language description that mention brand names, use-cases, and distinguishing features.

ArXiv (OGB-arXiv) [32]. Drawn from the Open Graph Benchmark, this citation graph contains 169,343 preprints connected by 1.17 M directed edges. Every node stores the full paper title plus abstract (roughly 140 – 300 tokens), resulting in the longest and most technical text fields in our study.

Amazon Products [11]. At 2.5 M nodes and 61 M edges, Amazon Products is the largest graph in our suite. Each node aggregates the product title, description, curated keyword tags, and representative review quotes. After basic cleaning this yields a median of 28 tokens and a long-tail distribution up to several hundred.

E Additional Settings

For completeness, we report the exact hyper-parameter values used in all experiments.

Subgraph text selection. The subgraph size $g = 4$, compression ratio $k/|D| = 0.6$. And GPT-2 [27] is used as the language model ϕ .

Attribute similarity matching. The learnable step-size parameter α is initialized to $5e - 3$. Optimization is carried out for 5,000 epochs with a mini-batch size of $m = 2000$ using stochastic gradient descent (SGD) with momentum 0.9 [28]. The learning rates for α and for the synthetic node attributes are $2e - 6$ and 100, respectively. To facilitate training, we utilize a pre-trained BERT model and keep it frozen and train the teacher model for 15 epochs.

Experiment environment. All experiments were conducted on a workstation running Ubuntu 22.04.3 LTS. The system was equipped with an Intel Core i9-13900K CPU, 64 GB of DDR5 RAM, and two NVIDIA GeForce RTX 4090 GPUs, each with 24 GB of VRAM. Our models were implemented in Python 3.11 using the PyTorch 2.5 framework, with CUDA 12.1 utilized for GPU acceleration.

F Limitation

While subgraph text selection is effective for compression, the resulting merged texts may sometimes lack human-interpretable semantic coherence. Additionally, the process of aligning attribute similarity matrices, which involves complex structural relationships, highlights the need for further research into more computationally efficient matching loss functions.